Department of Computer Science and Engineering
Computer Network Lab (CS39006)

Mini Project 2: SimpleMail – A Command-Line Mail System

Overview

In this project, you will build a complete command-line email system from scratch using C socket programming. The system consists of two programs: **smserver** (the SimpleMail server) and **smclient** (the SimpleMail client). The server stores and manages mailboxes for registered users, while the client connects to the server to send and retrieve mail. Communication between the client and server follows two custom protocols defined in this document: **SMTP2** (SimpleMail Transfer Protocol 2) for sending mail, and **SMP** (SimpleMail Pickup Protocol) for retrieving mail. Both protocols are text-based and operate over TCP.

The server listens on a single configurable port and distinguishes between SMTP2 and SMP sessions based on a handshake at the start of each connection. The server must be capable of handling multiple clients concurrently using `select()`/`poll()`. All mailboxes are stored as flat files on disk.

Part 1: The Server: smserver

Startup and User Registry

The server is launched as follows:

./smserver <port> <userfile>

The `userfile` is a plain text file where each line contains a username and password separated by a single space. Usernames are case-insensitive and consist only of lowercase alphabetic characters (a–z), with a maximum length of 20 characters. Passwords are case-sensitive alphanumeric strings with a maximum length of 30 characters. An example userfile might look like this:

```
alice secretpass1
bob hunter2
charlie x9Kp2mW
```

On startup, the server reads this file into memory. If the file is missing or malformed, the server prints an error and exits. For each user, the server must ensure a mailbox directory exists. Mailboxes are stored in a directory called `mailboxes/` relative to the server's working directory, with one subdirectory per user (e.g., `mailboxes/alice/`). Each email in a mailbox is stored as a separate

text file named with a monotonically increasing integer starting from 1 (e.g., `1.txt`, `2.txt`, etc.). You must track the next available ID per user so that IDs never repeat, even after deletion.

Connection Handling

When a client connects, the server sends a greeting line:

```
WELCOME SimpleMail v1.0
```

The server then waits for the client to declare what kind of session it wants. The client sends either `MODE SEND` or `MODE RECV`. The server responds with `OK` if it recognizes the mode, or `ERR Unknown mode` if it doesn't. After mode selection, the connection follows the corresponding protocol (SMTP2 for SEND, SMP for RECV). If the client sends no mode declaration within 30 seconds of connecting, the server closes the connection.

Server Console Output

The server should print a log line to stdout for every significant event: a new connection, mode selection, a mail being delivered, a mailbox being accessed, authentication success/failure, and disconnection. The format is up to you, but every log line must be prefixed with a timestamp in the format `[YYYY-MM-DD HH:MM:SS]`.

Part 2: SMTP2: The Sending Protocol

SMTP2 is used when the client has selected `MODE SEND`. This protocol allows the client to compose and deliver a mail to one or more recipients on the server. There is no authentication required to send mail; anyone can send mail to any registered user, but the server will reject mail addressed to users who don't exist.

SMTP2 Session Flow

After mode selection, the client and server engage in the following dialogue. All commands and responses are single lines terminated by `\r\n`. The maximum length of any single line in SMTP2 is 512 bytes including the terminator.

Step 1: Sender Declaration. The client sends a `FROM` command specifying the sender's display name. This is a free-form string (not necessarily a registered user) that will appear in the "From" header of the delivered mail. The format is:

```
FROM <display_name>
```

The display name can contain spaces and any printable ASCII characters. The server responds with `OK Sender accepted`.

Step 2: Recipient Specification. The client sends one or more `TO` commands, each specifying a registered username on the server:

```
TO <username>
```

For each `TO` command, the server checks whether the username exists in the user registry. If the user exists, the server responds with `OK Recipient accepted`. If not, it responds with `ERR No such user`. The client may send as many `TO` commands as it wishes. At least one recipient must have been accepted before the client can proceed to the next step.

Step 3: Subject. The client sends a `SUB` command with the subject line of the mail:

```
SUB <subject_text>
```

The server responds with `OK Subject accepted`. The subject may be empty (i.e., the client sends just `SUB` with nothing after it), in which case the subject is stored as "(no subject)".

Step 4: Body Transmission. The client signals the start of the mail body by sending the command `BODY`. The server responds with `OK Send body, end with CRLF.CRLF`. The client then sends the body of the email as multiple lines of text. To signal the end of the body, the client sends a line containing only a single period (`.`). This is known as the dot-termination convention. If a line in the actual body text starts with a period, the client must prepend an additional period to that line (byte-stuffing), and the server must strip the leading extra period when storing the mail. The maximum total size of the body after de-stuffing is 65536 bytes. If this limit is exceeded, the server discards the body and responds with `ERR Body too large`.

Step 5: Delivery Confirmation. After the dot-terminator is received and the body is within size limits, the server stores the mail in each accepted recipient's mailbox. Each stored mail file has the following exact format:

```
From: <display_name>
To: <comma-separated list of accepted recipients>
Subject: <subject_text>
Date: <YYYY-MM-DD HH:MM:SS>
---
<body text>
```

The `Date` field is the server's local timestamp at the time of delivery. The `---` line separates headers from the body. After successful storage, the server responds with `OK Delivered to N mailboxes` where N is the count of accepted recipients. If no recipients were accepted (all `TO` commands failed), the server instead responds with `ERR No valid recipients` when it receives the `BODY` command, and does not accept body text.

Step 6: Session End. After delivery, the client may send another `FROM` command to start composing a new mail within the same session, or it may send `QUIT` to end the session. The server responds to `QUIT` with `BYE` and closes the connection.

At any point during the SMTP2 session, if the client sends a command that is out of sequence (e.g., sending `BODY` before any `TO` command, or sending `SUB` before `FROM`), the server responds with `ERR Bad sequence`. The expected sequence is strictly: FROM → one or more TO → SUB → BODY → (body lines) → dot → delivery, and then either a new FROM or QUIT.

Part 3: SMP: The Retrieval Protocol

SMP is used when the client has selected `MODE RECV`. This protocol allows the client to authenticate as a registered user and then list, read, and delete mails in their mailbox.

SMP Authentication

Immediately after mode selection, the server sends:

```
AUTH REQUIRED
```

The client must then authenticate by sending its credentials in a single command using a simple challenge-response scheme. When the server sends `AUTH REQUIRED`, it also appends a space-separated random 8-character alphanumeric token called the **nonce**. So the full line from the server looks like:

```
AUTH REQUIRED <nonce>
```

The client must respond with:

```
AUTH <username> <hash>
```

The `hash` is computed as follows: concatenate the password with the nonce (password first, nonce second, no separator), then compute the **DJB2 hash** of the resulting string, and send the hash as an unsigned decimal integer. The DJB2 hash function is defined as:

```
unsigned long djb2(const char *str) {
    unsigned long hash = 5381;
    int c;
    while ((c = *str++))
        hash = ((hash << 5) + hash) + c;  /* hash * 33 + c */
    return hash;
}
```

The server performs the same computation using the stored password for the given username and compares the result. If the hashes match, the server responds with `OK Welcome <username>`. If the username doesn't exist or the hash doesn't match, the server responds with `ERR`

`Authentication failed`. The client gets at most 3 attempts to authenticate. After 3 failures, the server sends `ERR Too many failures` and closes the connection.

This challenge-response mechanism ensures that the raw password is never sent over the network.

SMP Mailbox Commands

After successful authentication, the client can issue the following commands in any order:

LIST: The client sends `LIST` and the server responds with a multi-line listing of all mails in the user's mailbox. The first line of the response is `OK <count> messages`, where count is the number of mails. Then, for each mail, the server sends one line in the format:

<id> <from_display_name> <subject> <date>

The fields are separated by a tab character (`\t`). After all listing lines, the server sends a line containing only `.` (dot-termination, same convention as SMTP2). If the mailbox is empty, the server sends `OK 0 messages` followed immediately by the dot line.

READ <id>: The client sends `READ` followed by a mail ID. If the mail exists, the server responds with `OK` on the first line, then sends the entire contents of the mail file verbatim (using dot-stuffing for any body lines starting with a period), terminated by a dot line. If the mail ID doesn't exist, the server responds with `ERR No such message`.

DELETE <id>: The client sends `DELETE` followed by a mail ID. If the mail exists, the server deletes it from disk and responds with `OK Deleted`. If it doesn't exist, the server responds with `ERR No such message`. Deleted mail IDs are never reused.

COUNT: The client sends `COUNT` and the server responds with `OK <count>` where count is the number of mails currently in the mailbox. This is a lightweight alternative to LIST when the client just wants to know if there's new mail.

QUIT: The client sends `QUIT` and the server responds with `BYE` and closes the connection.

If the client sends any unrecognized command, the server responds with `ERR Unknown command`.

Part 4: The Client: smclient

The client is launched as follows:

```
./smclient <server_ip> <port>
```

Upon connecting and receiving the server greeting, the client presents an interactive menu to the user:

Connected to SimpleMail server.

```
1. Send a mail
2. Check my mailbox
3. Quit
>
```

Sending a Mail (Option 1)

When the user selects option 1, the client initiates a `MODE SEND` session with the server. It then prompts the user interactively for each field:

```
From (your name): _
To (recipient username, empty line to finish): _
Subject: _
Body (type '.' on a line by itself to finish):
_
```

The client collects multiple recipients by prompting repeatedly until the user enters an empty line. It must ensure at least one recipient is provided before proceeding. The client handles the SMTP2 protocol dialogue transparently; the user never sees the raw protocol commands or responses. Instead, the client displays friendly messages like "Mail delivered to 2 recipients." or "Error: user 'dave' does not exist on this server." After delivery, the client returns to the main menu. For this, the client sends `QUIT` to close the SMTP2 session and the underlying TCP connection, then reconnects fresh when the user picks another option.

Checking the Mailbox (Option 2)

When the user selects option 2, the client initiates a `MODE RECV` session. It prompts for the username and password:

```
Username: _
Password: _
```

The client receives the nonce from the server, computes the DJB2 hash as described, and sends the `AUTH` command. If authentication fails, the client informs the user and allows retrying (up to the server's limit of 3 attempts). Upon successful login, the client displays a mailbox menu:

```
Mailbox for alice (3 messages)
1. List all messages
2. Read a message
3. Delete a message
4. Logout
>
```

The "List all messages" option issues a `LIST` command and displays the results in a clean tabular format. The "Read a message" option prompts for a message ID and displays the full mail content. The "Delete a message" option prompts for a message ID and confirms deletion. "Logout" sends `QUIT` and returns to the main menu.

Part 5: Protocol Summary

For quick reference, here is a compact summary of all protocol commands.

SMTP2 commands (client → server): `FROM <name>`, `TO <user>`, `SUB <subject>`, `BODY`, `QUIT`.

SMTP2 responses (server → client): `OK ...` for success, `ERR ...` for errors, `BYE` for session end.

SMP commands (client → server): `AUTH <user> <hash>`, `LIST`, `READ <id>`, `DELETE <id>`, `COUNT`, `QUIT`.

SMP responses (server → client): `OK ...` for success, `ERR ...` for errors, `AUTH REQUIRED <nonce>`, `BYE` for session end.

Both protocols use `\r\n` as the line terminator and `.` on a line by itself as the multi-line content terminator.

Part 6: Sample Execution Sequence

Below is a complete walkthrough showing exactly what the user sees in the terminal. Server-side log output is shown separately.

Server Terminal

```
$ ./smserver 9000 users.txt
[2026-03-15 10:00:00] Server started on port 9000
[2026-03-15 10:00:00] Loaded 3 users from users.txt
[2026-03-15 10:00:22] New connection from 127.0.0.1:54312
[2026-03-15 10:00:22] Client selected MODE SEND
[2026-03-15 10:00:45] Mail delivered from "Alice Smith" to [bob] (1 recipient)
[2026-03-15 10:00:45] Client disconnected
[2026-03-15 10:00:52] New connection from 127.0.0.1:54320
[2026-03-15 10:00:52] Client selected MODE RECV
[2026-03-15 10:00:58] Authentication successful for user bob
[2026-03-15 10:01:05] User bob READ message 1
[2026-03-15 10:01:12] User bob DELETE message 1
[2026-03-15 10:01:15] Client disconnected
```

Client Terminal: Sending a Mail

```
$ ./smclient 127.0.0.1 9000
Connected to SimpleMail server.
1. Send a mail
2. Check my mailbox
3. Quit
> 1

From (your name): Alice Smith
To (recipient username, empty line to finish): bob
  -> Recipient 'bob' accepted.
To (recipient username, empty line to finish): dave
  -> Error: user 'dave' does not exist on this server.
To (recipient username, empty line to finish):

Subject: Meeting Tomorrow
Body (type '.' on a line by itself to finish):
Hi Bob,
Just a reminder about our meeting tomorrow at 3 PM.
Please bring the project report.
.

Mail delivered to 1 recipient.

1. Send a mail
2. Check my mailbox
3. Quit
> 3
Goodbye.
```

Client Terminal: Checking the Mailbox

```
$ ./smclient 127.0.0.1 9000
Connected to SimpleMail server.
1. Send a mail
2. Check my mailbox
3. Quit
> 2

Username: bob
Password: hunter2

Welcome, bob!

Mailbox for bob (1 message)
1. List all messages
2. Read a message
```

```
3. Delete a message
4. Logout
> 1

ID   From            Subject              Date
---  ----            -------              ----
1    Alice Smith     Meeting Tomorrow     2026-03-15 10:00:45

1. List all messages
2. Read a message
3. Delete a message
4. Logout
> 2

Enter message ID: 1

From: Alice Smith
To: bob
Subject: Meeting Tomorrow
Date: 2026-03-15 10:00:45
---
Hi Bob,
Just a reminder about our meeting tomorrow at 3 PM.
Please bring the project report.

1. List all messages
2. Read a message
3. Delete a message
4. Logout
> 3

Enter message ID: 1
Message 1 deleted.

1. List all messages
2. Read a message
3. Delete a message
4. Logout
> 4

Logged out.

1. Send a mail
2. Check my mailbox
3. Quit
> 3
Goodbye.
```

Part 7: Wire-Level Trace

To remove any ambiguity, below is the exact sequence of bytes exchanged over the TCP connection for the "send mail" session above. Each line ends with `\r\n` which is omitted here for readability but must be present in your implementation. `C:` denotes data sent by the client and `S:` denotes data sent by the server.

SMTP2 Session Trace

```
S: WELCOME SimpleMail v1.0
C: MODE SEND
S: OK
C: FROM Alice Smith
S: OK Sender accepted
C: TO bob
S: OK Recipient accepted
C: TO dave
S: ERR No such user
C: SUB Meeting Tomorrow
S: OK Subject accepted
C: BODY
S: OK Send body, end with CRLF.CRLF
C: Hi Bob,
C: Just a reminder about our meeting tomorrow at 3 PM.
C: Please bring the project report.
C: .
S: OK Delivered to 1 mailboxes
C: QUIT
S: BYE
```

SMP Session Trace

```
S: WELCOME SimpleMail v1.0
C: MODE RECV
S: OK
S: AUTH REQUIRED a3bK9xQ2
C: AUTH bob 7572886318025498
S: OK Welcome bob
C: LIST
S: OK 1 messages
S: 1  Alice Smith Meeting Tomorrow  2026-03-15 10:00:45
S: .
C: READ 1
S: OK
S: From: Alice Smith
```

```
S: To: bob
S: Subject: Meeting Tomorrow
S: Date: 2026-03-15 10:00:45
S: ---
S: Hi Bob,
S: Just a reminder about our meeting tomorrow at 3 PM.
S: Please bring the project report.
S: .
C: DELETE 1
S: OK Deleted
C: QUIT
S: BYE
```

The hash value `7572886318025498` in the trace above is the DJB2 hash of the string `hunter2a3bK9xQ2` (password concatenated with nonce). You should verify this with your own implementation of DJB2 as a sanity check.

Deliverables

You must submit the following: the source files `smserver.c` and `smclient.c` (additional helper files are permitted), a `Makefile` that produces the two executables `smserver` and `smclient`, a sample `users.txt` file with at least 5 users, and a brief `README.md` explaining your design choices such as how you handle concurrency, how you manage mailbox file IDs, and any assumptions you made.

Evaluation Criteria

Your project will be evaluated on correct implementation of both SMTP2 and SMP protocols as specified (including edge cases like dot-stuffing, empty subjects, out-of-sequence commands, and nonexistent users), proper use of TCP sockets and concurrent connection handling, robustness against malformed input and partial reads, clean separation between protocol logic and user interface in the client, code quality and readability, and the completeness of your README. Also the server and clients for individual students should be compatible. Note that you might need further assumptions/design requirements, which MUST be clearly stated in your README file and in your code.